

Architecture d'un projet Rust : Package / Crate / Module

Michaël CHLON

2026-06-20

Contents

1	Vue d'ensemble	1
2	Package	2
3	Crate	2
4	Module	2
4.1	Chemins (paths)	3
4.2	Visibilité	3
5	Schéma explicatif	3
6	Arborescence type d'un package	4
7	À retenir	4

1 Vue d'ensemble

Rust organise le code sur trois niveaux emboîtés :

Niveau	Rôle	Défini par
Package	Unité de distribution / build (Cargo)	Cargo.toml
Crate	Unité de compilation (1 arbre de modules)	un fichier racine (.rs)
Module	Unité d'organisation + visibilité	mot-clé mod

Relation d'inclusion :

Package	1+ Crates	arbre de Modules	items (fn, struct, enum...)
---------	-----------	------------------	-----------------------------

2 Package

- Plus grande unité gérée par **Cargo**.
- Décrit par un seul `Cargo.toml` (nom, version, dépendances, édition).
- Contient **au moins une** crate, **au plus une** crate *library*.
- Peut contenir **plusieurs** crates *binary*.

Règles de Cargo (convention = racines de crate automatiques) :

Chemin	Crate produite
<code>src/main.rs</code>	crate binary (nom du package)
<code>src/lib.rs</code>	crate library (nom du package)
<code>src/bin/*.rs</code>	crates binary supplémentaires

3 Crate

- **Unité de compilation** : le compilateur `rustc` traite une crate à la fois.
- Deux types :
 - **binary** : possède `fn main()`, produit un exécutable.
 - **library** : pas de `main`, produit du code réutilisable (`.rlib`).
- Chaque crate a un **module racine** (crate root) :
 - `main.rs` → module racine de la crate binary
 - `lib.rs` → module racine de la crate library
- La crate forme un **arbre de modules** dont la racine s'appelle `crate`.

4 Module

- Unité d'**organisation** et de **contrôle de visibilité** (pub).
- Déclaré avec `mod`. Par défaut tout est **privé** (visible seulement dans le module et ses descendants).
- Trois façons de définir le contenu d'un module :

```
// 1. inline
mod jardin {
    pub mod legumes { pub fn planter() {} }
}

// 2. fichier frère : src/jardin.rs
mod jardin;

// 3. dossier : src/jardin/mod.rs (ou src/jardin.rs + src/jardin/legumes.rs)
mod jardin;
```

4.1 Chemins (paths)

- **absolu** : part de `crate` $\rightarrow$ `crate::jardin::legumes::planter()`
- **relatif** : `self`, `super` (module parent), ou nom direct.
- **use** crée un raccourci (import) dans le scope courant.

```
use crate::jardin::legumes::planter;
planter();
```

4.2 Visibilité

Mot-clé	Portée
(rien)	privé au module
<code>pub</code>	public
<code>pub(crate)</code>	public dans toute la crate seulement
<code>pub(super)</code>	public pour le module parent

5 Schéma explicatif

```

                                PACKAGE
                                (Cargo.toml)
nom · version · édition · deps

                                contient
```

```
CRATE library
src/lib.rs
(crate root)
```

```
CRATE binary
src/main.rs
fn main() {...}
```

arbre de modules

```
crate (racine)
  mod reseau          (src/reseau.rs)
    pub fn connecter()
  mod client          (src/reseau/client.rs)
    pub fn envoyer()
  mod jardin          (src/jardin/mod.rs)
    pub mod legumes
      pub fn planter()
```

Accès depuis main.rs :

```
use ma_lib::reseau::connecter; // crate externe (la library du package)
crate::jardin::legumes::planter(); // chemin absolu interne
```

6 Arborescence type d'un package

```
mon_package/
Cargo.toml      ← définit le PACKAGE
src/
  lib.rs        ← racine de la CRATE library
  main.rs       ← racine d'une CRATE binary
  reseau.rs     ← MODULE reseau
  reseau/
    client.rs   ← MODULE reseau::client
  bin/
    outil.rs    ← CRATE binary supplémentaire
  tests/        ← crates d'intégration (1 crate par fichier)
```

7 À retenir

- Cargo.toml = 1 package.
- 1 package = 0..1 library + 0..N binaries.
- 1 crate = 1 compilation = 1 arbre de modules avec racine `crate`.

- mod découpe, pub expose, use raccourcit, les paths localisent.